

12. Distributed Unique ID Generator

Interviewer: Every row in our system — orders, messages, events — needs a **unique ID**. We're way past a single database now: hundreds of app servers, sharded databases, multiple regions, all minting IDs concurrently. Design an ID generator that hands out **globally unique 64-bit IDs** at massive scale, with no central bottleneck, and — ideally — IDs that sort roughly by creation time. How do you build it?

Candidate: This is a fun one because the "obvious" answers — auto-increment, UUID — both break down at scale for different reasons, and the interesting part is *why*. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions before I design: 1. Do IDs need to be **numeric/compact** (fits a DB `bigint`, sorts well in an index), or is a 128-bit string like a UUID acceptable? 2. Does "roughly time-ordered" matter — i.e., do newer IDs need to be numerically larger, for pagination or index locality — or is pure uniqueness enough? 3. How many independent ID-minting nodes are we talking about, and what's peak IDs/sec *per node*? 4. Can we tolerate a tiny bit of clock dependency (NTP-synced clocks), or must this work with zero assumptions about wall-clock time?

Interviewer: Compact — must fit a 64-bit integer, it's going straight into DB primary keys. Yes, time-ordering matters a lot — we paginate and index on ID. Assume ~2,000 app nodes across 3 regions, each needing up to 50,000 IDs/sec at peak. Clocks are NTP-synced; assume mostly-correct but occasionally-skewed clocks, not adversarial ones.

Candidate: Good, that's a real design brief. Let me pin down requirements.

Functional - `nextId()` → a 64-bit integer, **globally unique** across every node, every region, forever. - IDs are **monotonically increasing over time** (not strictly per-node, but "later in time ⇒ numerically larger" in the common case) — useful for DB primary keys and time-range queries. - Thousands of nodes mint IDs **independently and concurrently**, no shared request per ID.

Non-functional — this is where it's interesting - **No central bottleneck** — a "give me an ID" call to one server, for every ID, becomes the single point of contention and failure for the entire system's writes.

- **Extreme per-node throughput, near-zero latency** — a local computation, not a network hop. -

Compact: 64 bits, not 128 — going into indexes, compared billions of times. - **Resilient to clock skew** — clocks drift, get NTP-corrected, or occasionally jump backward; we must never produce a duplicate or wildly out-of-order ID because of that. - **No coordination on the hot path** across 2,000 nodes in 3 regions.

2. Estimation — the number that shapes the design

Candidate: Let me size the throughput, because it tells me how many bits I can "afford" to spend on each part of the ID.

2,000 nodes, peak 50,000 IDs/sec/node → fleet peak = $2,000 * 50,000 = 100M$ IDs/sec.

If I reserve a per-millisecond sequence counter of B bits per node:

capacity/node = 2^B IDs per millisecond = $2^B * 1,000$ IDs/sec.

Need $2^B * 1,000 \geq 50,000 \Rightarrow 2^B \geq 50 \Rightarrow B \geq 6$ bits ($2^6=64/\text{ms}=64,000/\text{s}$).

Give real headroom → 12 bits = 4,096 IDs/ms/node = 4,096,000 IDs/sec/node.

50,000 < 4,096,000 → ~80x headroom per node, comfortably absorbs spikes.

Worker-id space: 2,000 nodes need distinct identities.

10 bits = 1,024 (too few); 12 bits = 4,096 (comfortable headroom) → pick ~12-13 bits, split further across regions (see multi-region section).

Timestamp: need decades of life.

41 bits of milliseconds since a custom epoch (not 1970) = 2^{41} ms ≈ 69.7 years.

Budget check: 1 (sign) + 41 (time) + 12 (worker) + 10 (seq) = 64 bits exactly if I trim sequence to 10 bits ($1,024/\text{ms/node} = 1,024,000/\text{sec/node}$) → still >> 50,000 needed.

So there's room to trade bits between worker-id and sequence depending on fleet size.

Candidate: So the real question isn't "can we generate unique numbers" — it's **how do we pack timestamp + node-identity + a local counter into 64 bits so that no two nodes, anywhere, ever produce the same value, without any of them talking to each other per ID.**

3. V1 approaches — compare before committing

Interviewer: Before you jump to a fancy scheme — why not just use a UUID? Or a database auto-increment column?

Candidate: Let me walk through the obvious options and where each one dies, because the comparison is the whole story here.

- A) UUID v4 (random 128-bit)
 - + zero coordination – any node, any time, no network, no clock needed
 - 128 bits → 2x the storage/index size of a bigint
 - NOT time-ordered: fully random → constant B-tree page splits and poor cache locality on insert (a well-known Postgres/MySQL pain point at high write volume)
 - rejected: fails "compact + sortable" outright.
- B) DB auto-increment (single sequence)
 - + trivially unique, trivially ordered
 - ALL 100M IDs/sec funnel through one sequence on one DB → global bottleneck and single point of failure for every write, at a throughput no single row/sequence can sustain
 - rejected: reintroduces exactly the bottleneck we're eliminating.
- C) Ticket server (central range allocator)
 - + removes the DB hit from the PER-ID path – a node asks a central server for a BLOCK (e.g., "[50000–59999]") and hands IDs out locally afterward
 - still a shared dependency, hit less often; a crash mid-allocation burns the unused tail of a block (a gap – usually acceptable)
 - cross-node ordering is coarse: whichever node grabbed which block
 - better than B, but still a startup/refill-time dependency with approximate ordering.
- D) Snowflake-style: timestamp + worker-id + sequence packed into 64 bits
 - + zero network calls per ID – pure local arithmetic
 - + roughly time-ordered (timestamp is the high bits, so it dominates)
 - + compact – exactly 64 bits, fits a bigint
 - needs a reasonably correct clock per node
 - needs a ONE-TIME worker-id assignment mechanism (not per ID)
 - satisfies every requirement simultaneously.

Candidate: Snowflake wins because it's the only approach that is *simultaneously* coordination-free on the hot path, compact, and time-ordered. UUID gives up compactness and ordering; DB auto-increment gives up horizontal scale entirely; a ticket server is a good partial answer but still has a shared dependency and coarser ordering.

4. The Snowflake design — bit layout (the centerpiece)

Interviewer: Okay, sell me on the bit layout. Walk me through exactly how a single 64-bit integer encodes all of this.

Candidate: Here's the layout, most-significant bit first:

64 bits total:

1 bit	41 bits	12 bits	10 bits
0	timestamp (ms since epoch)	worker id	sequence
bit 63 sign, always 0 (keeps ID positive in signed types)	bits 62-22 milliseconds since a CUSTOM epoch (not 1970) e.g. 2024-01-01 → 2 ⁴¹ ms ≈ 69.7 yrs before wraparound	bits 21-10 which node minted this ID (0 - 4,095)	bits 9-0 per-ms counter, reset to 0 each new millisecond, incremented per ID in that ms (0 - 1,023)

Field-by-field: - **Sign bit:** always 0 — keeps the value a positive signed 64-bit int (Java `long`, SQL `bigint`), avoiding negative-number surprises. - **Timestamp (41 bits):** milliseconds since a *custom* epoch chosen at launch (not Unix 1970) — this maximizes usable years. Because it's the highest-order field after the sign bit, **the timestamp dominates the numeric value** — as real time moves forward, this field only grows, so later IDs are numerically larger. That's the entire mechanism behind "roughly time-ordered." - **Worker id (12 bits):** 4,096 possible node identities, assigned once at process startup (see §6). This is what lets thousands of nodes mint IDs with **zero runtime coordination** — each owns a disjoint slice of the ID space. - **Sequence (10 bits):** a per-millisecond, per-node counter, reset to 0 every new millisecond. $2^{10} = 1,024$ IDs/ms/node = **1,024,000 IDs/sec/node** — comfortably above our 50,000/sec requirement, with room to spare.

Generation algorithm (per node, no locks needed across nodes):

```
on nextId():
    now = current_time_ms()
    if now < last_ms:                # clock moved backward! see §8
        handle_clock_skew()
    if now == last_ms:
        seq = (seq + 1) mod 1024
        if seq == 0:                # this ms's budget is exhausted
            wait_until_next_ms(); now = current_time_ms()
    else:
        seq = 0
    last_ms = now
    return (now << 22) | (worker_id << 10) | seq
```

Why this beats a naive counter: the only mutable, contended state is `(last_ms, seq)` — and it's **in-memory, on one process**, guarded by an atomic compare-and-swap or a lock held for nanoseconds. There is no cross-node shared state at all, because each node's worker-id slice makes its output space disjoint from every other node's by construction.

5. Multi-region — the datacenter bits

Interviewer: You've got 3 regions now, each running its own pool of nodes. How do you keep worker ids globally unique across regions without a cross-region call?

Candidate: Split the 12-bit worker-id field itself into a **datacenter/region sub-field** and a **per-node sub-field within that region** — a static, pre-agreed partitioning, decided at design time, not at runtime.

12-bit worker-id field, split:

4 bits	8 bits
dc id	node id in dc
0-15 datacenters	0-255 nodes per datacenter

us-east: dc=0 -> worker ids 0*256..0*256+255 (0-255)
eu-west: dc=1 -> worker ids 256-511
ap-south: dc=2 -> worker ids 512-767
(13 datacenter slots reserved for future growth)

- Each **region owns a disjoint slice** of the worker-id space by static configuration — **zero cross-region coordination, ever**, not at startup and not per ID. Global uniqueness holds because no two regions can ever hand out the same (dc, node) pair.
- **Time-ordering across regions is only approximate**, not exact: each region's clock is independently NTP-synced, so an ID minted in eu-west at time T could be a hair smaller or larger than one minted in us-east at a true wall-clock instant very close to T, due to clock drift (typically single-digit ms). "Globally unique, roughly time-ordered" is the promise — not "linearizable across continents." Strict global ordering would need a logical clock or a single ordering authority, which directly fights the "no coordination" goal.

6. Worker-id assignment — the one thing that DOES need coordination

Interviewer: Where does a node get its (dc, node id) pair from? That sounds like exactly the kind of shared state you said we were avoiding.

Candidate: It's the one necessary exception, and the key is that it's **used only once, at startup — never on the per-ID hot path.**

ZooKeeper (or etcd/Consul) — touched ONLY when a node boots, not per ID:

node starts in dc=eu-west
-> creates an EPHEMERAL znode under /workers/eu-west/{pid-or-hostname}
-> claims the lowest unclaimed integer in [0, 255] for that dc
-> caches that worker id in local memory for the process's lifetime
-> from this point on, nextId() never touches ZooKeeper again

node crashes / network partition:
-> its ephemeral znode disappears when its session expires
-> the worker id becomes claimable again by a new node
-> ZK's session model guarantees no two live nodes ever hold the same znode -> no two nodes can claim the same worker id
(no split-brain double-claim)

Why not just assign worker ids from a config file or a random pick? A human-managed config doesn't scale to 2,000 nodes across autoscaling groups and is exactly the kind of thing that gets

duplicated by a copy-paste deploy mistake — silently causing duplicate IDs. A random pick from 4,096 slots has a real collision probability once you have hundreds of nodes (birthday problem) unless something enforces exclusivity. ZooKeeper's ephemeral-znode-per-session model gives us a hard mutual-exclusion guarantee at negligible cost, since it's hit only at startup.

7. Latency, consistency, and the async question

Interviewer: Where does the latency go for a single `nextId()` call? And where's the queue-and-async-workers layer you've had in every design so far?

Candidate: Both answers are the same point, actually. `nextId()` reads the local clock, does a compare-and-swap on the in-memory `(last_ms, seq)` pair, and does bit arithmetic — sub-microsecond, no I/O, no network. There's deliberately **no async layer on this path**: adding a queue would only add latency to something whose entire value is *not having any*.

The only things that happen off this path are genuinely peripheral: worker-id registration with ZooKeeper (once, at startup — not "async," just not on the hot path at all), and fire-and-forget metrics/alerts on clock-skew events, published to a monitoring pipeline so they never block generation.

Uniqueness/consistency guarantee, precisely: - **Strong, across nodes:** two nodes can never collide, because their worker-id fields differ (assigned exclusively via ZooKeeper) — every other field can match and the output still won't. - **Strong, within one node:** the `(last_ms, seq)` CAS serializes concurrent callers on the same process in nanoseconds, since it never leaves memory. - **Eventual/best-effort:** cross-node and cross-region *ordering* at sub-millisecond granularity, because of clock drift. Uniqueness is absolute; ordering is only approximate at the margins — worth saying out loud, since people often assume Snowflake gives perfect global ordering, and it doesn't.

```
[ App node ]
| nextId() <- local, in-memory, no network, no queue
▼
[ (last_ms, seq) in-process state ] — CAS, nanoseconds

[ App node ] — (once, at boot) → [ ZooKeeper ] — claim worker id
    (the ONLY network hop in the whole design, and it never repeats per ID)
```

8. Failure modes: clock skew and machine-id conflicts

Interviewer: Two failure scenarios I want you to walk through: the node's clock jumps *backward*, and two nodes somehow think they own the same worker id.

Candidate:

Clock moves backward (NTP step correction, VM pause/resume, hypervisor migration):

```

if now < last_ms:
    -> the node must NOT mint an ID with an earlier timestamp than one it
        already issued – that risks both breaking ordering AND colliding
        with a sequence value it already used at that same (or a later)
        apparent ms.
    Option A (safest): REFUSE to generate – raise an alarm, block nextId()
        until now >= last_ms again. Small availability hit, zero risk.
    Option B (pragmatic): if the skew is tiny (a few ms), SLEEP until the
        clock catches up to last_ms, then proceed normally.
    Mitigate at the source: run NTP in SLEW mode (gradual correction) on
        all fleet nodes, not STEP mode (instant jump) – this makes backward
        jumps rare and, when they happen, tiny.

```

Worker-id collision (two nodes believe they own the same id — e.g., a ZK session bug or a misconfigured static-assignment fallback):

```

-> CATASTROPHIC if it happens: two nodes can mint byte-identical IDs at
    the same millisecond + sequence value.
-> prevented structurally, not detected after the fact: ZooKeeper
    ephemeral znodes tie a worker-id claim to a live session; ZK
    guarantees no two sessions hold the same znode. A crashed node's
    znode disappears only once its session truly expires, so there's no
    window where two live processes both believe they own id 47.
-> defense in depth: log the worker id per process and run a periodic
    audit ("are two live processes reporting the same id?") since the
    blast radius of a coordinator bug (silent duplicate primary keys)
    is severe even though it's rare.

```

ZooKeeper itself goes down: only affects NEW nodes trying to start and claim a worker id — already-running nodes are unaffected, since the worker id is cached in memory with zero per-ID dependency on ZK. Core principle: the coordination path failing must never touch the hot (generation) path.

Sequence overflow within a millisecond (>1,024 IDs requested in 1ms on one node): the node busy-waits/sleeps for the next millisecond tick, then resets sequence to 0 — a sub-millisecond stall, not an error, and given our 50,000/sec requirement against a 1,024,000/sec/node ceiling, this should essentially never trigger in practice.

9. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
ID scheme	Snowflake (64-bit: timestamp + worker id + sequence)	Local, lock-free generation; compact; roughly time-ordered. <i>Not UUID v4</i> — 128 bits, no ordering, kills B-tree index locality on inserts. <i>Not DB auto-increment</i> — one sequence can't absorb 100M IDs/sec and is a single point of failure. <i>Not a ticket server</i> — still a shared dependency hit periodically, and cross-node ordering is coarser.
Per-ID hot path	In-process library call, no network	The entire value proposition is a local computation; any per-ID network hop reintroduces the exact bottleneck we're eliminating.
Worker-id coordination	ZooKeeper (or etcd/Consul), ephemeral znodes, startup-only	Gives a hard mutual-exclusion guarantee (no two live nodes can claim the same id) via session-scoped ephemeral nodes, at a cost that's paid once per node lifetime, not per ID. <i>Not a static config file</i> — doesn't scale to autoscaling fleets and is one copy-paste mistake

Concern	Choice	Why this, and why not the alternative
		away from a silent duplicate-id disaster. <i>Not random assignment</i> — collision probability becomes real at hundreds of nodes (birthday problem) with no exclusivity guarantee.
Clock source	Local system clock, NTP in slew mode	Local clock reads are free and fast; slew-mode NTP avoids the backward time jumps that cause the one real hazard in this design. <i>Not a logical/vector clock</i> — massive complexity for a benefit (perfect cross-node ordering) we don't actually need.
Cross-region uniqueness	Static datacenter-id sub-field, partitioned at design time	Zero cross-region network calls, ever — each region owns a disjoint worker-id slice by config. <i>Not a global coordinator across regions</i> — would add cross-continent latency/dependency to a path that must stay purely local.
External access (non-JVM clients)	Thin gRPC service wrapping the same library, if unavoidable	Binary framing + persistent HTTP/2 beats REST for this latency-sensitive, high-QPS internal call. This reintroduces a network hop, so it's the fallback, not the default.

The one-sentence justification you'd say out loud: "Snowflake packs a millisecond timestamp, a worker id, and a per-node sequence into one 64-bit integer so that every node generates IDs purely locally — no network call, no shared counter, no database — while the timestamp field's dominance in the bit layout keeps IDs roughly sorted by creation time; the only coordination in the whole system is a one-time, ZooKeeper-guaranteed worker-id claim at startup, which is exactly why the hot path stays contention-free even at 100 million IDs per second across a 2,000-node, 3-region fleet."

10. Follow-up curveballs

Interviewer: Your fleet grows to 20,000 nodes. The 12-bit worker-id field only has 4,096 slots. What now?

Candidate: Caught **at design time**, not patched at runtime — bit widths are baked into every ID ever issued, so you can't silently widen a field without breaking comparability with old IDs. Size generously up front (e.g., reserve 14-16 bits for worker id if this growth is foreseeable) or, if already over budget, borrow bits from the sequence field — trading per-node throughput headroom for more node identities, which is fine given the ~80x headroom we had.

Interviewer: Two services need IDs sorted with plain byte-order string comparison (e.g., as a partition key in Cassandra or S3). A raw int64 doesn't sort correctly as a string unless you're careful. How do you handle that?

Candidate: Encode the same bit layout as a **fixed-width, zero-padded string** (base 32/36, or zero-padded decimal) so lexicographic string ordering matches numeric ordering — the idea behind ULID. The underlying timestamp+worker+sequence scheme doesn't change, only the external representation.

Interviewer: Why not just have each DB shard use its own auto-increment, with odd/even (or mod-N) ranges assigned per shard?

Candidate: Works at small, fixed scale, but every node needs the *full* partitioning scheme, and adding/removing a shard means re-coordinating it fleet-wide — a harder membership-change problem. Snowflake's worker-id space sidesteps this: a new node just claims an unused id from a large pre-reserved pool and starts minting immediately.

Interviewer: How would you debug "which node generated this ID, and roughly when," given just the 64-bit number?

Candidate: A nice side benefit of plain arithmetic over an opaque hash: bit-shift and mask the value apart — `timestamp = id >> 22`, `worker_id = (id >> 10) & 0xFFF`, `sequence = id & 0x3FF` — then add the custom epoch back to get a real wall-clock time. No lookup table needed; the ID is self-describing.

11. Deep-dive: "Why not just use UUIDv4 everywhere?"

Interviewer: Snowflake feels like a lot of machinery — bit budgets, worker-id leases, clock-skew handling. Why not skip all of it and just call `uuid4()` on every node? It's already unique, it's a one-liner, and every language has it built in.

Candidate: It *is* unique, and for a lot of systems that's the whole requirement — but for a **primary key that lives in a B-tree index**, random UUIDs are actively hostile, for two compounding reasons.

Why randomness specifically hurts a database

Sequential-ish key (Snowflake / auto-increment):

```
insert 1001 -> goes to the rightmost/"hot" leaf page, already in cache
insert 1002 -> same page, appended
insert 1003 -> same page, appended
... page fills -> ONE clean split at the end, then repeat
    [ page N-1 : full, cold ] [ page N : hot, being appended to ]
```

Random UUIDv4 key:

```
insert 7f3a... -> lands in a random page, anywhere in the tree
insert 2c91... -> a DIFFERENT random page
insert e04b... -> yet another random page
    [ page 1 ] [ page 2 ] [ page 3 ] ... [ page N ]
    ^ every insert can touch a page that's cold (evicted from
      buffer pool) and only partially full -> that page splits
      sooner and more often, and each split copies ~half the
      page's rows into a new page
```

Effect: instead of ~1 split per full page (sequential), you get frequent splits on MANY partially-full pages scattered across the whole index -> far more page writes per row inserted ("write amplification"), worse buffer-cache hit rate (every insert can be a cache miss), and a bloated, fragmented index.

This is a well-documented, measurable problem on Postgres and MySQL/InnoDB at high write volume — it's not theoretical. On top of the locality issue, there's a second, simpler cost: **128 bits vs. 64**. Every row's primary key, and every foreign key referencing it, and every index entry pointing to it, is now twice the size — more bytes on disk, more bytes in the buffer pool, more bytes crossing the network in replication.

Comparison table

Concern	UUIDv4 (random)	DB auto-increment	Snowflake	UUIDv7
Size	128 bits	typically 32/64 bits	64 bits	128 bits
Coordination per ID	none	one shared sequence (bottleneck)	none (local, after one-time worker-id claim)	none
Index locality on insert	terrible — fully random, scattered page writes	perfect — always appends at the end	good — timestamp-dominant, mostly appends	good — timestamp-dominant, mostly appends
Time-sortable	no	yes (but only because it's centralized)	yes, to the millisecond	yes, to the millisecond
Horizontal scale (many independent minters)	excellent	none — single point of contention	excellent	excellent

Concern	UUIDv4 (random)	DB auto-increment	Snowflake	UUIDv7
Guessable / enumerable	no (128 bits random)	yes — sequential, leaks row count	no (worker-id + seq are structured but not trivially guessable)	no (random low bits)
Self-describing (decode time/origin)	no	no	yes — bit-shift and mask	yes

When UUIDv4 is actually fine

Candidate: I wouldn't ban UUIDv4 — I'd scope it. It's the right choice when: - The ID is **not the clustering/primary key of a hot, high-write B-tree** — e.g. an opaque external-facing token, an idempotency key, a session id, or a secondary attribute stored in a non-clustered/hash index. - You explicitly **want unguessability** — UUIDv4's randomness is a security property (no one can enumerate `id+1`), which a sequential or timestamp-prefixed ID gives away. - Write volume is low enough that page-split overhead is noise, or the store isn't a B-tree at all (e.g. a hash-partitioned KV store doesn't care about insert locality the way Postgres does).

Rule of thumb: *If the ID is the thing your database clusters rows by and you're writing a lot, sortable beats random — use Snowflake or UUIDv7. If the ID just needs to be unique and unguessable and isn't driving index layout, UUIDv4 is simpler and perfectly fine.*

12. Deep-dive: clock skew — when a node's clock moves backward

Interviewer: Walk me through, in detail, exactly how a backward clock jump can produce a duplicate or non-monotonic ID on one node — not just that it "might happen," but the precise sequence of events.

Candidate: This is the sharpest failure mode in the whole design, because the algorithm's one safety invariant — "the sequence counter for a given millisecond is only ever used once per node" — is built entirely on the assumption that `current_time_ms()` never goes backward.

The timeline

Node's local clock, worker_id = 47, sequence resets to 0 each new ms:

wall-clock	node sees	event	emits
t=100	ms=1000	first ID this ms, seq=0	(1000,47,0)
t=100.3	ms=1000	same ms, seq=1	(1000,47,1)
t=100.6	ms=1000	same ms, seq=2	(1000,47,2)
		last_ms is now recorded as 1000	
...			
		[NTP daemon fires a STEP correction: clock was 8ms fast, it steps BACKWARD to fix it]	
t=100.9	ms=993	<-- clock now reads EARLIER than last_ms! if we naively do: now(993) != last_ms(1000) -> "new ms" -> seq=0 we emit (993, 47, 0)	

PROBLEM 1 – non-monotonic: (993,47,0) is numerically SMALLER than the (1000,47,0/1/2) already emitted and possibly already stored/indexed. Any consumer relying on "newer ID > older ID" (pagination, tie-breaking, causality) now sees an ID appear to travel backward in time.

PROBLEM 2 – potential duplicate: suppose the clock keeps running forward normally from 993 and a few ms later legitimately reaches 1000 again (t=101.7, ms=1000). Naively: now(1000) == last_ms(1000)? Only true if last_ms wasn't updated correctly in between – but if it WAS updated to 993 and forward again, by the time wall-clock time genuinely re-reaches 1000, seq resets to 0 and we emit (1000,47,0) – IDENTICAL to the very first ID we emitted at the top of this timeline. Same (timestamp, worker_id, sequence) triple, twice. That is a byte-for-byte duplicate ID.

The root cause in both cases is the same: the algorithm's uniqueness proof implicitly assumes `now()` is monotonically non-decreasing on each node. A STEP-mode NTP correction, a VM pause/resume, a hypervisor live-migration, or (rarely) a leap-second handling bug all break that assumption for a brief window.

Ranked fixes

Candidate:

1. **Refuse to generate and wait until the clock catches up (safest).** If `now < last_ms`, don't emit anything — block `nextId()` (or return a retryable error) until `now() >= last_ms`. Zero risk of duplication or reordering; the cost is a short availability gap on that one node, bounded by however far back the clock jumped (usually single-digit milliseconds with slew-corrected NTP, but STEP corrections can be larger and unbounded in the worst case).
2. **Borrow from a monotonic clock, not wall-clock time.** Use the OS's monotonic clock source (e.g. `CLOCK_MONOTONIC` on Linux, which never jumps backward due to NTP/leap-seconds — only wall-clock/ `CLOCK_REALTIME` does) to detect elapsed time, and only use wall-clock time to *anchor* the epoch once at startup. This sidesteps the NTP-step hazard entirely for the hot-path clock reads, at the cost of needing to periodically reconcile monotonic-elapsed time with wall-clock for cross-node comparability.
3. **Track `last_ms` as a hard floor, independent of what the clock says.** Whatever `now()` returns, never let the emitted timestamp field go below `last_ms`. If `now < last_ms`, *pin* the emitted

timestamp to `last_ms` and keep incrementing the sequence within that pinned millisecond (falling through to sequence-exhaustion/wait logic if it overflows) rather than accepting the clock's smaller value outright. This trades a small amount of drift between "ID timestamp" and "true wall-clock time" for a hard monotonicity guarantee, and doesn't require blocking the way option 1 does.

4. **Machine-id assignment safety is a separate but related guarantee.** None of the above matters if two live nodes can hold the *same* worker id — that turns "one node's clock glitch" into "two nodes silently colliding forever." The ZooKeeper ephemeral-znode lease from §6 is what makes worker-id collision structurally impossible (session-scoped exclusive claim), so it belongs in the same defense-in-depth story as clock-skew handling, not as an alternative to it.

Recommended approach

Candidate: I'd combine **#1 and #3**: treat `last_ms` as a monotonic floor the generator never regresses below (fix #3), but if the gap is large enough that pinning would mean generating a large burst of same-millisecond IDs against a stale timestamp, fall back to **refusing and waiting** (fix #1) past some threshold (say, >5ms of skew) rather than pinning indefinitely. Pair this with **NTP in slew mode fleet-wide** (already called out in §8) so large STEP corrections — the actual trigger — become rare in the first place, and with **structural worker-id safety via ZooKeeper (#4)** so a clock glitch on one node can never combine with a worker-id collision to produce a truly catastrophic duplicate. Monitor and alert on every backward-clock event even when handled correctly — it's a signal something in the fleet's NTP configuration needs attention.

What to say out loud: *"The algorithm's uniqueness guarantee silently assumes the clock never goes backward. I make `last_ms` a hard floor — the emitted timestamp never regresses — and fall back to blocking `nextId()` briefly if the skew is too large to pin through safely. That, plus slew-mode NTP to make big jumps rare, plus ZooKeeper making worker-id collisions structurally impossible, covers both the transient clock hazard and the permanent two-nodes-same-identity hazard with two independent, complementary mechanisms."*

Summary — the mental model

The distributed ID generator problem is really: **"give thousands of independent nodes a way to mint globally-unique, roughly time-ordered numbers without ever talking to each other or a shared server per request."** UUIDs solve uniqueness but give up compactness and ordering; DB auto-increment and ticket servers solve ordering but reintroduce a shared bottleneck. The Snowflake layout — **sign bit + 41-bit timestamp + worker-id bits + sequence bits, packed into one 64-bit integer** — resolves all three constraints at once by making the timestamp the dominant field (so IDs sort by time) and giving every node a disjoint identity slice (so uniqueness needs zero runtime coordination). The only coordination in the entire system is a **one-time, ZooKeeper-guaranteed worker-id claim at startup**; the one real operational hazard is **clock skew**, handled by refusing-or-stalling on backward jumps and running NTP in slew mode. Multi-region just means slicing the worker-id space further by datacenter — still zero cross-region calls, at the cost of only *approximate* ordering across regions.